From Knowledge Graphs to Constraint Hypergraphs

Joe Gregory
University of Arizona
1127 E James E Rogers Way
Tucson, AZ 85704
joegregory@arizona.edu

John Morris
Clemson University
328A Fluor Daniel Engineering Innovation Building
Clemson, SC 29631
jhmrrs@clemson.edu

Abstract

As systems engineering increasingly depends on digital models and integrated data environments, the graph representation of engineering knowledge has become a key enabler of semantic consistency, traceability, and automation. To implement this, we may use ontology languages such as the Web Ontology Language (OWL2DL) and the Ontological Modeling Language (OML) to define the concepts and relations within our domain, and then use logical reasoners to infer new information and detect inconsistent, incomplete or incorrect information. However, a limitation of purely logical representations is their inability to perform mathematical reasoning. Logical inference can identify missing or inconsistent facts, but it cannot compute or optimize quantitative parameters that drive engineering analyses.

This paper presents the development of an approach that bridges this gap by generating solvable constraint hypergraphs (CHGs) in an executable format from knowledge graphs (KGs) that capture systems engineering knowledge. Ontologies provide the formal semantics of the relevant domains, while data instances populate those models with project-specific information. From this structured knowledge, mathematical relations among parameters are automatically identified and transformed into a CHG representation in which nodes represent quantities and hyperedges represent the mathematical relations between them.

In this paper, we describe an end-to-end workflow from ontological modeling and data integration to constraint extraction and solution, and we demonstrate its application through a spacecraft design example. In this example, a KG describing a spacecraft and its environment is used to automatically generate the mathematical relations that describe power, mass, and communication performance. The resulting constraint hypergraph is then solved to derive consistent parameter values across the system model.

By translating logical relations into executable constraint structures, this approach enables automated numerical reasoning without compromising semantic rigor. It reduces the risk of inconsistencies when defining equations manually and strengthens the integration between logical and analytical representations within digital engineering environments.

Keywords

Digital engineering, semantic web technologies, knowledge graph, constraint hypergraph, analysis.

Introduction

The development of complex engineering systems increasingly demands efficient and reliable methods to integrate and manage data across the system lifecycle. Digital Engineering (DE) is emerging as an approach to address this challenge, leveraging the digital thread to ensure seamless connectivity of system data from design to deployment (Zimmerman et al., 2019). By integrating traditionally siloed tools

and processes, DE aims to reduce the risks associated with manual data handling, such as the use of outdated information and human error. Industry leaders prioritize digital thread initiatives, particularly for their ability to enable traceability across requirements, models, analyses, and test results, thereby fostering informed decision-making and improved system performance (Dertien & Hastings, 2021).

In particular, seamless traceability from requirements to models to simulation results is becoming an increasingly important aspect of this digitalization effort Buffoni et al., 2017; Mengist et al., 2021; Otter et al., 2015. To support this, data integration and management technologies are crucial. Semantic Web Technologies (SWTs) have the potential to significantly enhance the capabilities of DE by structuring information in a way that enables insight to be automatically attained through reasoning and querying (Patel & Jain, 2021). These technologies, which include ontologies, reasoners, and graph-based query languages, enable the representation of complex relationships across multiple domains in a modular, interoperable manner. In particular, ontologies provide a shared conceptual framework that integrates domain knowledge with raw data, enabling cross-domain interoperability and facilitating the validation of knowledge (Gruber, 1991, 1993). By capturing system-related information in a configuration-managed repository, SWTs improve data consistency and enhance the ability to query and analyze highly connected data efficiently (Medhi & Baruah, 2017). We refer to a graph representation of our data that has been structured according to a relevant set of ontologies as a knowledge graph (KG).

While SWTs enable reasoning and querying of KGs based on the semantics of the domains in which we are interested, their reasoning capabilities are fundamentally symbolic rather than numerical. The Web Ontology Language (OWL2DL), for example, reflects a decidable fragment of first-order logic. This means that there is an algorithm which can determine in a finite number of steps the truth-value of any statement expressed in the language (Beverley et al., 2025). It cannot, however, be used to solve mathematical equations. This limits the utility of the language in an engineering context, where engineering analyses depend on quantitative relationships between parameters. Constraint Hypergraphs (CHG), by contrast, are computable networks of mathematical functions. Each node represents a variable or parameter, and each hyperedge encodes a constraint linking them.

In this paper, we make three arguments. First, we argue that KGs and CHGs provide complementary representations of a dataset in a way that enables both description (via rigorous semantics) and analysis (via computable mathematics). Second, we argue that we often have enough information within our KG to automatically generate a corresponding CHG. Third, we argue that the approach of automatically generating a CHG from a KG is of significant benefit to the engineer as it reduces the risks associated with manually building analysis models from descriptive models.

In Section 2, we describe the relevant concepts in more detail. In Section 3, we define the research methodology adopted. The overall approach that we have developed is presented in Section 4 and is applied to an example in Section 5. Limitations of the work and possible next steps are discussed in Section 6 and the work is concluded in Section 7.

Background

This research builds upon work produced previously by researchers at the University of Arizona (UA) related to DE and its application. This includes work on the development of a digital environment to support students and DE research (J. Gregory et al., 2024), the development of an ontology stack to support systems engineering (J. Gregory et al., 2025), and the development of ontologies to support test and evaluation (J. Gregory & Salado, 2024) and other use cases (J. R. Gregory et al., 2025). Throughout this work, we have used the Ontological Modeling Language (OML) as the basis for this work. OML is an extension to OWL2DL developed as part of the OpenCAESAR initiative at the Jet Propulsion Laboratory (JPL) (Elaasar, 2019). OML enables reification of relations and convenient use of mixin classes as aspects (Wagner et al., 2020). Data is captured in OML descriptions. OML also uses the concept of 'bundles' to allow the modeler to group together multiple descriptions into a closed-world dataset that can be reasoned upon, validated and queried. The closed-world assumption allows the reasoner to highlight the absence of expected information. From these OML descriptions, the OML Rosetta tool can build a triple database in Resource Description Framework (RDF) format (OpenCAESAR, 2023; World Wide Web Consortium, 1997). This enables the use of SPARQL Protocol and RDF Query Language (SPARQL), a semantic query language that can be used to retrieve and ma-

nipulate data stored in RDF databases (Pérez et al., 2009; World Wide Web Consortium, 2013).

This research in semantic representation schema is paralleled by efforts to define executable models that encode how a system will behave. The basis of these efforts is work by Willems (Willems, 2007) who showed that behaviors are constraints on the state space of the system. Deterministic behaviors (that effect the system's state space) can be modeled as functions, whose domains are a set of states that constrain the value of another. These functions can be composed together to reveal interactions on the global system; for instance, suppose that the maximum thrust T of a rocket can be predicted by volume V of available fuel and fuel volume is constrained by the rocket's height h. Each variable could be assigned any value from the set of real numbers $\mathbb{R}$, and is represented as a set. A model of these two relations could consequently assign two functions f_1 and f_2 mapping V to T and h to V. Another function f_3 could be formed by composing f_1 with f_2. These composed function describes how the height of the rocket affects the rocket's thrust by mapping h to T.

This model can be depicted graphically by giving each state variable as a node and each function as an edge, as in the example in Figure 1. Due to the arity of a function being of any dimension, these edges become hyperedges, making the graph a hypergraph of behavioral constraints. Constructed in this manner, the CHG depicts how all the states in a system are related to each other. The graphical nature of a CHG represents function composition as paths in the hypergraph, or, in other words, each path in the hypergraph represents a function that constrains a state in the system to the values of several other states.

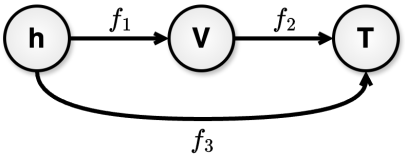

Figure 1. Basic CHG showing functions constraining the thrust T of a rocket to the volume of fuel V and V to the rocket's height h

Because all behaviors can be represented as functions, CHGs are universal modeling framework into which any system model–be it bond graph, Petri net, discrete-event, or algebraic–can be deconstructed (Morris, Mocko, & Wagner, 2025). The collection of state variables gives all the information that can be known about the system. Furthermore, by providing composition via path formation, the CHG is able to encode each simulation that can be performed on the system, with the simulation inputs and outputs given by the path function's domain and codomain respectively. Because of this, the action of executing a model becomes the work of finding paths through the hypergraph connecting known information (input nodes) to unknown information (output nodes). This research makes use of ConstraintHg: open-source software that automates this pathfinding process to engender flexible, system-wide simulation (Morris, 2025).

Here we present an example of how the KG and CHG can be integrated together. Engineers (and humans more generally) often have an understanding of the relationship between domain knowledge and the analyses that can be performed using that knowledge. In other words, if we have a graph representation of our own domain knowledge, then we should have the capability to construct a meaningful CHG that represents the relevant mathematical relations between the parameters in that model. In fact, we rely on domain experts to do exactly this all the time. In this example, we assume we know the following:

- we have a system $\mathbf{S}$, with mass attribute $\mathbf{m_S}$
- we have components $\mathbf{C_1}$ to $\mathbf{C_n}$
- these components have mass attributes $\mathbf{m_{C_1}}$ to $\mathbf{m_{C_n}}$ respectively.
- system $\mathbf{S}$ comprises components $\mathbf{C_1}$ to $\mathbf{C_n}$

The mass of the system can be calculated as the sum of the mass of the system components, i.e.:

$$\mathbf{m}_S = \sum_{i=1}^{n} m_{C_i} \tag{1}$$

Traditionally, an engineer would manually review the engineering data to identify the (possibly many) instances of this composition pattern and then manually construct corresponding 'sum' functions to support the multiple analyses of the mass properties of the system. By representing the four bullet points above in a KG, however, we can define a computational rule that will identify the existence of all instances of this pattern and construct the functions accordingly–and combine them into a solvable CHG. Note that by following this approach, we only have to define the mapping between this semantic pattern and the corresponding mathematical function once.

This has two major potential benefits: **1)** we do not need to define every *instance* of a particular function, saving time and reducing the risk of mistakes, and **2)** we do not have to manually define a particular workflow–a CHG will be generated and then *all* functions for which we have a complete set of inputs can be automatically executed. i.e., all quantity values that it is possible to calculate will be calculated.

Methodology

The research presented in this paper has been based on the Design Science Research Methodology (DSRM). Design science research is "motivated by the desire to improve the environment by the introduction of new and innovative artifacts and the processes for building these artifacts" (A. R. Hevner, 2007). The DSRM is a methodology that has been developed to enable design science research. It can be divided into five activities (identification of problem, definition of objectives, development of solution, demonstration of solution, and evaluation of solution) (A. Hevner & Chatterjee, 2010) with the flows and feedback loops between these activities forming three distinct cycles (A. R. Hevner, 2007). The *relevance cycle* relates the research to a relevant problem; the *rigor cycle* ensures that the solution is built on an appropriate knowledge base; the *design cycle* iterates between building and evaluating the solution. The DSRM, as applied to this research, is presented in Figure 2. The five tasks described in this section correspond to the five DSRM activities.

Task 1: we identify the limitations of existing approaches. These limitations have been described in Section 1 and Section 2. We also create an example dataset that can be used to evaluate the solution. We have created an example dataset that describes a spacecraft and its orbit. We describe this example in more detail in Section 5.

Task 2: we define the objectives of the solution. The objectives for the solution that are covered in this paper are as follows:

1. the solution shall detect instances of patterns in the KG and, according to the mappings defined, generate instances of the corresponding functions;
2. the solution shall combine these functions into a single, solvable CHG;
3. the solution shall solve for all possible values for a given set of input quantities.

Note that we believe that the potential of this approach extends far beyond the objectives identified here–these initial objectives are the subject of this particular paper. We discuss possible future objectives in Section 6.

Task 3: we develop the solution. This includes selection of appropriate modeling languages and tools. In previous work, we have used OML to capture our knowledge and our analyses have been built on Python (J. Gregory et al., 2025). To remain consistent with our previous efforts, we have used the same languages here. We then develop the solution itself. This involves development of the relevant ontologies, identification and creation of mappings from patterns in our KG to functions in our CHG, creation of a Python script that can generate a CHG from the combined set of functions, and the implementation of a Python-based CHG solver. The solution is presented in Section 4.

Task 4: we apply the solution to the example dataset. We first create an OML representation of the spacecraft dataset that was created in Task 1. Then, we create three different versions of this dataset in which different subsets of the quantities are known. We then apply the solution developed in Task 3 to these three datasets to determine whether we can generate and solve an appropriate CHG. This application example is presented in Section 5.

Task 5: we evaluate the results of Task 4. We discuss whether the objectives have been met and whether the limitations presented in Section 1 and Section 2 have been addressed. This discussion is presented in Section 6.

KG to CHG Approach

In this section, we present our proposed solution. The solution itself is comprised of OML ontologies, SPARQL queries, and Python scripts. We describe each of these in the following subsections. All of these products are licensed as open source products available through a GitHub repository (J. Gregory & Morris, 2025).

Ontologies

This is the output of Task 3a (see Figure 2). To implement this solution, practitioners require an ontological representation of both the domain that they are interested in modeling and the functions that will be used in the CHG. Note that in this and later sections

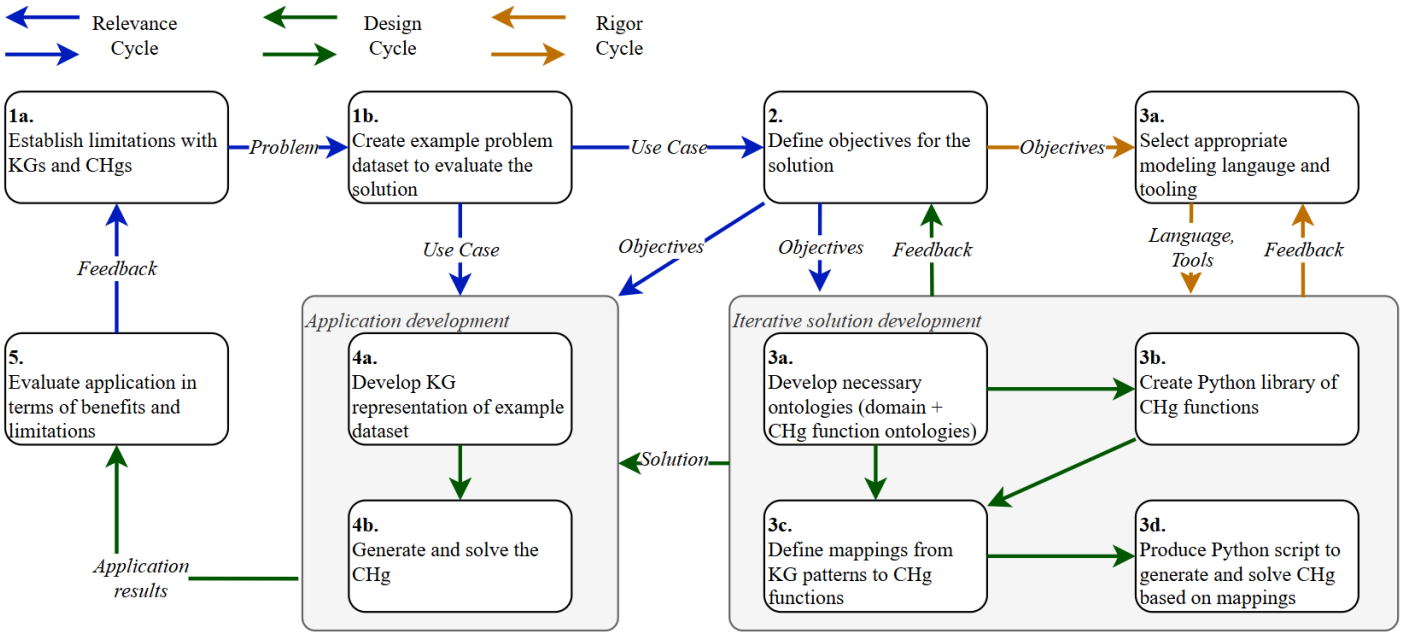

Figure 2. Design science research methodology applied to this research

we use the convention of denoting an 'OML Model' (which includes both ontologies and dscriptions) using single quotation marks, an ontological **Class** using bold text, an ontological *relation* using italicized text, and an individual using underlined text.

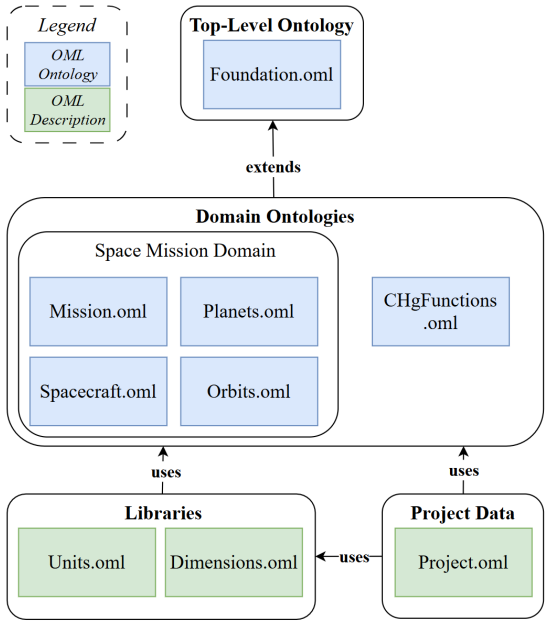

Figure 3. Ontology stack developed to support the spacecraft example

For this paper, our goal has been to investigate our approach by applying it to a spacecraft model. We therefore require a set of ontologies that will enable us to define our spacecraft mission. For this example, we have created four domain ontologies ('Mission', 'Planets', 'Spacecraft', and 'Orbits'). Note that we would eventually incorporate these domain ontologies into our overall ontology stack (the UAOS), but for now we have demonstrated our approach using a small ontology stack for simplicity. We have also created a simple Top-Level Ontology (TLO) called 'Foundation', which is essentially a simplified version of the 'Basic Formal Ontology' (BFO) (Arp et al., 2015). BFO is a TLO commonly used to support engineering applications (Karray et al., 2021). 'Foundation' supports interoperability between the domain ontologies by defining high-level classes and relations used by all domain ontologies (e.g., **MaterialEntity**, **Quantity**, *hasName*). The ontology stack is presented in Figure 3.

We also define another domain ontology: 'CHGFunctions'. This ontology contains ontological representations of all functions that we will use. We do not need to define these functions mathematically–we will define the math elsewhere. Instead, we just need to define the number of arguments (and whether the order of the arguments is important) and the number of returned values. These classes will act as the bridge from our space mission domain ontologies to the relevant Python functions. The OML representations of the sum function (**F_sum**) and subtraction function (**F_sub**) are presented in Figure 4. Note that the argument order is important for **F_sub** ($a - b \neq b - a$), but is not important for **F_sum** ($a + b = b + a$).

```
concept F_sum < Function [
    restricts hasArgument to min 2 Argument
    restricts hasReturn to exactly 1 Return
]

concept F_sub < Function [
    restricts hasArgument0 to exactly 1 Argument
    restricts hasArgument1 to exactly 1 Argument
    restricts hasReturn to exactly 1 Return
]

concept F_fspl < Function [
    restricts hasArgument0 to exactly 1 Argument
    restricts hasArgument1 to exactly 1 Argument
    restricts hasReturn to exactly 1 Return
]
```

Figure 4. OML representation of summation, subtraction and free space path loss (FSPL) functions

Analysis Libraries

To enable practitioners to automatically generate CHGs from KGs and solve them, we will refer to libraries containing generic analyses that can be instantiated as needed. These analyses can range in complexity from simple algebraic functions, through numerical approximation functions, to complex functions that leverage domain-specific solvers (e.g., finite element analysis). Some of these are general and can be applied to a wide range of applications (e.g., summation, subtraction, differentiation) while others are more specific to a particular application (e.g., calculation of free space path loss for telecommunications across space).

For the spacecraft example presented in this paper, we will define a set of Python-based functions. Table 1 lists the functions relevant to this example. They have been defined in Python, with the definitions for the **F_sum**, **F_sub** and **F_fspl** (free space path loss) functions given as:

```
def F_sum(**kw) -> float:
    """Return the sum of all keyword input values."""
    return sum(kw.values())

def F_sub(**kw) -> float:
    """Return first minus second keyword value."""
    vals = list(kw.values())
    if len (vals) < 2:
        raise ValueError ("needs at least two inputs.")
    return vals[0] - vals[1]

def F_fsp1(**kw) -> float:
    """Calculates Free Space Path Loss in decibels."""
    vals = list(kw. values())
    if len(vals) < 3:
        raise ValueError ("R_fsplCalc needs 3 inputs")
    Altitude_km = float(vals[0])
    Freq_Hz = float(vals[1])
    Radius_km = float(vals [2])
    Freq_MHz = Freq_Hz / 1e6 #Hz to MHz
    MaxSlantDistance_km = math.sqrt(2 * Radius_km *
        Altitude_km)
    FSPL_dB = 32.45 + 20.0 * np.log10(MaxSlantDistance_km
        ) + 28.0 * np.log10(Freq_MHz)
    return FSPL_dB
```

Mappings from KG to CHG

In order to automate the generation of the relevant CHG from a KG, we need to define the mappings from the semantics captured in our domain ontologies to the functions defined in the libraries. One such example of this has been provided in the Background section: if we have an instance of the class **System** that *hasQuantity* some **Mass**, and *hasComponent* multiple instances of **Component**, each of which *hasQuantity* some **Mass**–then we know that we can represent the relation between the values of these mass instances via a summation function.

Another example is provided graphically in Figure 5. In this example, a pattern that relates Free Space Path Loss (FSPL), a quantity of the mission, to other quantities in the ontology has been defined. When this pattern is detected in a KG, an instance of the function **F_fspl** will be created along with the appropriate relations to its arguments and return value. Each mapping is defined as a SPARQL CONSTRUCT query. SPARQL CONSTRUCT queries are used to construct new instances of functions and their relations if a particular pattern is found in the KG. The SPARQL CONSTRUCT query displayed in Figure 6 is equivalent to the graphical representation of the instantiation of the **F_fspl** function in Figure 5.

This process of mapping these patterns in our KG to appropriate functions is something that engineers do all the time. Communications engineers, for example, know that if they detect the pattern shown on the left of Figure 5, they can use the function on the right to calculate FSPL. In some cases, the mapping may be considered trivial and thus may be done implicitly (e.g., summation of component masses). In other cases, the analysis may be more complex and the mapping may require more thought. The key differences of defining these mappings explicitly using SPARQL CONSTRUCT queries are twofold: **1)** the mapping is represented formally (i.e., we do not rely on the implicit knowledge of domain experts), thus supporting traceability; **2)** the mapping only needs to be defined *once* (rather than once per instantiation) and then all instantiations of the relevant function can be automatically generated.

Function Name	Description	Arguments	Return	Argument Order Importance	Application
F_sum	Sum input arguments	$a_1, a_2, ..., a_n$	$\sum_{i=1}^{n} a_i$	No	General
F_sub	Subtract one input argument from another	b, c	$b - c$	Yes	General
F_fspl	Calculate worst-case free space path loss (FSPL) for a spacecraft in orbit	frequency (Hz) altitude (km) planet radius (km)	FSPL (dB)	Yes	Space Mission
F_deorbit	Calculate time taken for a spacecraft in orbit to deorbit	spacecraft effective area (m^2) semi-major axis (km) spacecraft mass (kg)	deorbit time $(years)$	Yes	Space Mission
F_eirp	Calculate power received by receiving antenna	transmitter power (W) transmitter gain (dB) antenna gain (dBi) system losses (dB)	EIRP (dBm)	Yes	Space Mission
F_recPower	Calculate the Effective Isotropic Radiated Power (EIRP) of a comms subsystem	EIRP (dBm) FSPL (dB) receiver gain (dBi)	received power (dBm)	Yes	Space Mission

Table 1. List of functions that can be instantiated to construct a CHG by mapping quantities defined in the domain ontologies to function arguments

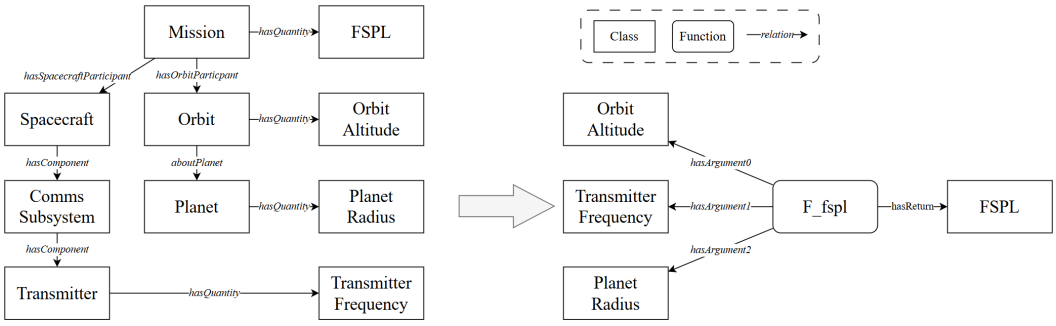

Figure 5. Free space path loss (FSPL): identifying the FSPL pattern in the KG and instantiating a function to calculate FSPL

```
PREFIX xsd:     <http://www.w3.org/2001/XMLSchema#>
PREFIX fou:     <http://uaontologies.com/spacecraft_ontology/Foundation#>
PREFIX sc:      <http://uaontologies.com/spacecraft_ontology/Spacecraft#>
PREFIX orb:     <http://uaontologies.com/spacecraft_ontology/Orbits#>
PREFIX mis:     <http://uaontologies.com/spacecraft_ontology/Mission#>
PREFIX pla:     <http://uaontologies.com/spacecraft_ontology/Planets#>
PREFIX fun:     <http://uaontologies.com/CHg_Ontology/Functions#>

CONSTRUCT {     ?fun a fun:F_fspl , fun:Function .
                ?fun fun:hasArgument0 ?altitude .
                ?fun fun:hasArgument1 ?frequency .
                ?fun fun:hasArgument2 ?radius .
                ?fun fun:hasReturn ?fspl .            }

WHERE {         ?mission fou:hasQuantity ?fspl .
                ?fspl a mis:FSPL .
                ?mission mis:hasSpacecraftParticipant ?sc .
                ?sc a sc:Spacecraft .
                ?sc fou:hasComponent ?com .
                ?com a sc:COM .
                ?com fou:hasComponent ?tra .
                ?tra a sc:Transmitter .
                ?tra fou:hasQuantity ?frequency .
                ?frequency a fou:Frequency .
                ?mission mis:hasOrbitParticipant ?orbit .
                ?orbit fou:hasQuantity ?altitude .
                ?altitude a orb:Altitude .
                ?orbit orb:aboutPlanet ?planet .
                ?planet fou:hasQuantity ?radius .
                ?radius a fou:Distance .
```

Figure 6. SPARQL CONSTRUCT query that will instantiate the F_fspl class when the relevant pattern is detected in the knowledge graph

CHG Construction and Solution

The mappings, defined using SPARQL CONSTRUCT queries, allow the user to reason across a dataset and generate a set of appropriate functions that relate the quantities within the dataset. In order to *solve* the corresponding CHG, we first need to combine these functions and generate a representation of the overall CHG that can be solved using ConstraintHg. The generation of the CHG is achieved in two steps: **1)** a SPARQL SELECT query (List_Functions.sparql) retrieves all instances of functions that have been constructed; **2)** a Python script (generate_CHG.py) ingests the result of this SPARQL SELECT query (List_Functions.json) and converts this into a format that can be solved by the CHG solver (generated_CHG.py).

It is now nearly possible to solve the CHG–but we first need a list of known values. The CHG re-

lates the quantities via functions but, depending on which values we know, it may only be possible to execute a portion of the CHG and solve a subset of the available functions. Of course, it is possible for our KG to contain quantities for which we do not know the associated value (e.g., we can define the quantity system_mass without knowing its value)–indeed it is these quantities that we wish to use the CHG to help us solve. The solution of the CHG is achieved in two steps: **1)** a SPARQL SELECT query (List_Parameters.sparql) retrieves all known values of the quantities defined in the CHG; **2)** a Python script (solve_CHG.py) ingests the results of this SPARQL SELECT query (List_Parameters.json) and solves the CHG (generated_CHG.py) using the ConstraintHg software and the analysis libraries that we defined earlier. That is to say, it traverses the CHG calculating all values that it is possible to calculate, and does so iteratively until a traversal of the CHG returns no new values. In this way, all values that it is possible to calculate given the initial set of known values are automatically calculated.

Application to Spacecraft Example

In this section, we step through an example. We present each step primarily from the user perspective while also explaining what is happening on the backend. The pipeline from a user perspective is displayed in Figure 7. As with the workflow engine, this example is available for download at (J. Gregory & Morris, 2025).

In this example, we define a spacecraft mission and use the 'Mission', 'Planets', 'Spacecraft', and 'Orbits' ontologies to structure our knowledge. In this case, we model our spacecraft mission information directly as OML–though in other work we have demonstrated how it is possible to capture system design information in domain-specific languages and tools (such as SysML v2, Jama Connect, Duro) and automatically generate the OML representation (J. Gregory et al., 2025; J. R. Gregory et al., 2025).

```
description <http://uaontologies.com/spacecraft_example/WildCatMission#> as WildCatMission {

    uses <http://www.w3.org/2001/XMLSchema#> as xsd
    uses <http://uaontologies.com/spacecraft_ontology/Foundation#> as fou
    uses <http://uaontologies.com/spacecraft_ontology/Mission#> as mis
    extends <http://uaontologies.com/spacecraft_example/Libraries#> as lib
    extends <http://uaontologies.com/spacecraft_example/WildCatSat#> as wcs
    extends <http://uaontologies.com/spacecraft_example/WildCatOrbit#> as wco

    // Mission Definition

    instance wildCatMission : mis:SpaceMission [
        mis:hasSpacecraftParticipant wcs:wildCatSat
        mis:hasOrbitParticipant wco:wcs_Orbit
        mis:hasGroundSegmentParticipant wcs_GroundSegment
        fou:hasQuantity wcm_deorbitTime
        fou:hasQuantity wcm_FSPL
    ]

    instance wcm_deorbitTime : mis:DeorbitTime

    instance wcs_GroundSegment : mis:GroundSegment [
        fou:hasQuantity wcs_GroundSegment_ReceiverGain
        fou:hasQuantity wcs_GroundSegment_Mass
        fou:hasQuantity wcs_GroundSegment_ReceivedPower
    ]

    instance wcs_GroundSegment_Mass : fou:Mass

    instance wcs_GroundSegment_ReceiverGain : mis:ReceiverGain

    instance wcs_GroundSegment_ReceivedPower : fou:PowerGen

    instance wcm_FSPL : mis:FSPL

    // Mission Measurements

    instance wcs_GroundSegment_ReceiverGain_Measurement : fou:GainMeasurement [
        fou:measurementOf wcs_GroundSegment_ReceiverGain
        fou:hasValue "7.8"^^xsd:double
        fou:hasUnit lib:dB
    ]
```

Figure 8. Partial OML representation of WildCat Mission

Our mission is wildCatMission, and it will involve a single spacecraft, wildCatSat. wildCatSat participates in a low-Earth orbit, wcs_Orbit, and it will communicate with operators via a ground segment, wcs_GroundSegment, located on Earth. wcs_GroundSegment comprises a single receiver, wcs_GroundSegment_Receiver. wildCatSat itself is made up of multiple subsystems, which each comprise multiple components. All of these physical elements have a mass, and some have an associated power draw. Some physical entities have other quantities associated with them. wcs_Transmitter, for example, has an associated transmitter frequency (wcs_Transmitter_Frequency) and the high power amplifier, wcs_HPA, has an associated gain (wcs_HPA_Gain). All of the quantities contained in the dataset are listed in Table 2. We capture this information across four OML descriptions: 'WildCatMission.oml', 'WilCatSat.oml', 'WildCatOrbit.oml' and 'Earth.oml' which are then bundled together and used to produce our RDF knowledge graph (see (OpenCAESAR, 2023; Wagner et al., 2020) for details on how OML manages descriptions). A snippet of 'WildCatMission.oml' is presented in Figure 8.

Once all of this information has been captured as a set of OML descriptions, we can run the analysis pipeline (CHG_pipeline.py), displayed in Figure 7, which steps through the capabilities described in the previous section. An informal representation of the resulting CHG is presented in Figure 9. Based on the mappings defined, ten instances of functions

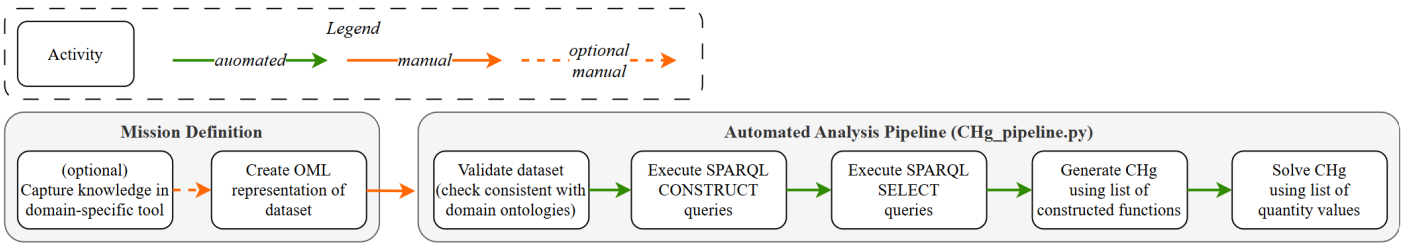

Figure 7. Pipeline from a user perspective (note that automated flows between activities are automatically triggered on completion of the initial activity)

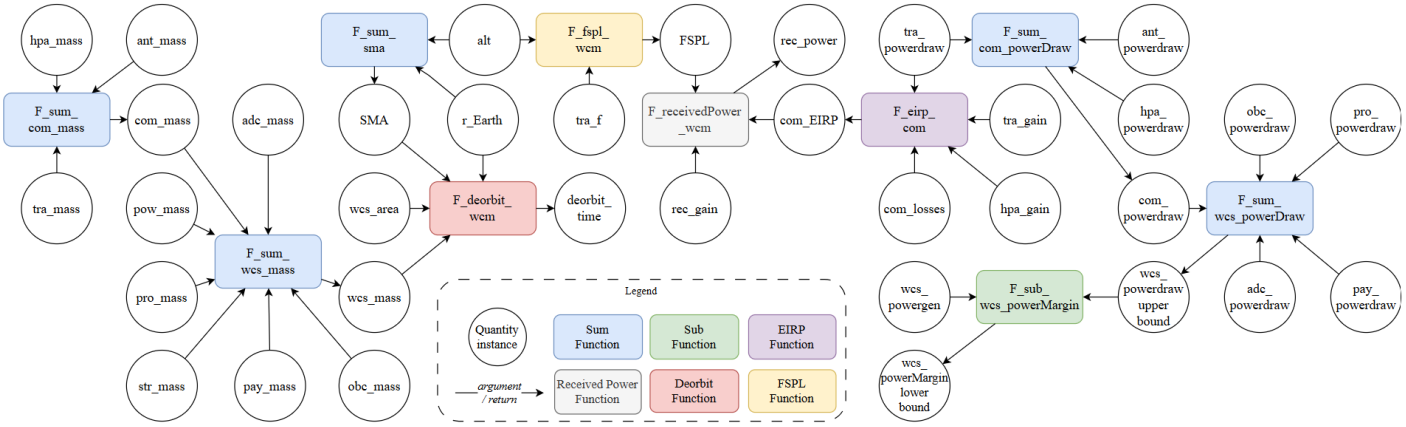

Figure 9. Informal representation of the constraint hypergraph that has been automatically constructed from the 'Wildcat Mission' knowledge graph and subsequently solved

have been instantiated. The CHG relates the quantities described in Table 2 via these functions. To explore the capabilities of the CHG solver, we ran this pipeline with three different sets of known quantity values (i.e., we added and removed values from the OML descriptions). The results of these analyses are also displayed in Table 2 under the column headers *Scenario 1 Values*, *Scenario 2 Values* and *Scenario 3 Values*. In Scenario 1, we included just enough information to enable the CHG to solve for all quantities for which we did not initially know the value. In Scenario 2, we made one change–we removed the transmitter power value (which was 3.1W in Scenario 1) and then solved the CHG. In this case, we see that this has a significant impact on the results. Five further quantities are affected as the effect propagates through the CHG. In this example, we have completely replaced the transmitter with a new type of transmitter (which has different values of mass, power draw, and transmission frequency). We updated the definition of the transmitter in the OML description, and reran the analysis pipeline. The impact of this modification on the spacecraft and its mission can be seen in Table 2.

Discussion and Future Work

While the authors believe that the capabilities presented in this paper have significant potential in the context of digital engineering, the application example presented has served as a proof of concept and the KG to CHG pipeline remains a work in progress. In this section, we discuss some of the required further work and identify some of the limitations of the current approach that need to be addressed.

In this example, the Python-based functions that are generated are only capable of providing solutions in the direction specified. If we instantiate a sum function ($a + b = c$), then the CHG will solve for c if we know the values of a and b. However, if we know the values of b and c, for instance, it would not be able to solve for a. The reduces the efficacy of the CHG. In future work, we will address this by integrating symbolic mathematics to represent engineering relations in a form that can be solved in any direction. This will allow each function in the model to support multi-directional inference, enabling the CHG to compute any variable given the others. One way that this can be practically implemented is by using the SymPy package (SymPy Development Team, 2025).

Currently, every time there is a change in the OML descriptions, we re-run the entire analysis pipeline

Quantity Name	Quantity Type	Unit	Quantity Of	Scenario 1 Values	Scenario 2 Values	Scenario 3 Values
adcMass	Mass	kg	Attitude Determination and Control subsystem	7.4	7.4	7.4
adcPowerDraw	PowerDraw	W	Attitude Determination and Control subsystem	4.2	4.2	4.2
antGain	Gain	dB	Antenna	8.6	8.6	8.6
antMass	Mass	kg	Antenna	1.1	1.1	1.1
antPowerDraw	PowerDraw	W	Antenna	0.2	0.2	0.2
comEIRP	Gain	dB	Comms subsystem	59.314	n/a	60.528
comLosses	Gain	dB	Comms subsystem	-2.3	-2.3	-2.3
comMass	Mass	kg	Comms subsystem	6.7	6.7	8.2
comPowerDraw	PowerDraw	W	Comms subsystem	17.5	n/a	18.5
earth_Radius	Distance	km	Earth	6371	6371	6371
groundSegment_ReceivedPower	Gain	dB	Ground segment	-99.644	n/a	-95.931
groundSegment_ReceiverGain	Gain	dB	Ground segment	7.8	7.8	7.8
hpaGain	Gain	dB	High Power Amplifier	18.1	18.1	18.1
hpaMass	Mass	kg	High Power Amplifier	2.0	2.0	2.0
hpaPowerDraw	PowerDraw	W	High Power Amplifier	14.2	14.2	14.2
obcMass	Mass	kg	On-Board Computer	1.0	1.0	1.0
obcPowerDraw	PowerDraw	W	On-Board Computer	8.7	8.7	8.7
payMass	Mass	kg	Payload	1.0	1.0	1.0
payPowerDraw	PowerDraw	W	Payload	8.7	8.7	8.7
powMass	Mass	kg	Power subsystem	6.4	6.4	6.4
powPowerGen	PowerGen	W	Power subsystem	45.0	45.0	45.0
proMass	Mass	kg	Propulsion subsystem	1.0	1.0	1.0
proPowerDraw	PowerDraw	W	Propulsion subsystem	5.1	5.1	5.1
strMass	Mass	kg	Structural subsystem	1.0	1.0	1.0
traFrequency	Frequency	Hz	Transmitter	2000000000	2000000000	1500000000
traMass	Mass	kg	Transmitter	3.6	3.6	5.1
traPowerDraw	PowerDraw	W	Transmitter	3.1	n/a	4.1
wcm_FSPL	Gain	dB	WildCat Mission	166.758	166.758	164.259
wcm_deorbitTime	Duration	yrs	WildCat Mission	3.597	3.597	3.817
wco_Orbit_Altitude	Distance	km	Wildcat Sat orbit	529	529	529
wco_Orbit_SMA	Distance	km	Wildcat Sat orbit	6900	6900	6900
wildCatSatEffectiveArea	Area	m^2	Wildcat Sat	12.5	12.5	12.5
wildCatSatMass	Mass	kg	Wildcat Sat	24.5	24.5	26.0
wildCatSatUpperBoundPowerDraw	PowerDraw	W	Wildcat Sat	38.2	n/a	39.2
wildCatSatLowerBoundPowerMargin	Power	W	Wildcat Sat	6.80	n/a	5.80

Table 2. List of quantities defined in the WildCat Mission knowledge graph and their values, after automatic generation and solution of the CHG. Initially known values are black. Values that have been calculated by the CHG are blue. Unknown values are red.

(see Figure 7). This pipeline is fully automated and, for the spacecraft example, completes in less than 20 seconds. However, the CHG solver currently iterates over the CHG multiple times–whenever a function is executed and a new value is calculated, the solver iterates over the whole CHG again to check whether any other functions can now be solved using this new value. This is not the most efficient way of ensuring that all possible values are calculated. In future work, we will explore ways to improve the solver efficiency.

One of the main efforts of the work presented in this paper has been to define the mappings from the KG to the CHG as SPARQL CONSTRUCT queries. Formally representing this translation from domain knowledge to mathematical relationships required significant thought and time. However, as more functions are defined and the libraries of analyses and mappings grow, we expect that this workload will reduce significantly and will continue to decrease with each new project.

The spacecraft example presented in this paper contained ten instantiations of six functions. Five of these functions are relatively simple algebraic equations. The deorbit function, defined in detail in (J. R. Gregory et al., 2025), is numerical and propagates the orbital trajectory over multiple time steps to estimate the deorbit time. All of these functions have been written in Python. However, it is possible to define functions that call more complex domain-specific solvers such as Finite Element Analysis (FEA) soft-

ware or a orbit propagation tool such as the General Mission Analysis Tool (GMAT) or Systems Tool Kit (STK). These APIs of these platforms are integrated into the CHG as new edges, allowing these more complicated relationships to be connected to the rest of the model (Morris, Indupally, et al., 2025). The implementation of such a distributed model ecosystem is planned for future work.

Similarly, the application presented in this paper has largely focused on the static analysis of a spacecraft mission, but the functions do not need to be restricted to the mission or system of interest. We may incorporate functions that calculate project-based metrics such as verification coverage and digital thread completeness (a measure of traceability). A particular use case that this approach may lend itself to is sensitivity analysis. It is possible to automate sensitivity analyses by developing a wrapper that can solve the CHG repeatedly while varying input values. These ideas will also be considered as we progress through this research.

Emphasize that the rules themselves need to be thought out more. These are just simple examples to demonstrate how the workflow works.

Conclusion

In this paper, we have presented the Knowledge Graph (KG) to Constraint Hypergraph (CHG) pipeline. The pipeline enables users to describe their system-of-interest in a KG and automatically generate a corresponding CHG that mathematically relates the quantities in the KG and will automatically solve for all possible unknown quantity values. We demonstrated this approach by applying it to an example dataset that describes a low-Earth orbit spacecraft mission. We applied this in three different scenarios, where a different set of quantity values was known in each scenario. In each scenario, a CHG was successfully generated and fully solved (i.e., all quantity values that were possible to calculate were calculated). The benefits of this work are that **1)** users do not need to define every *instance* of a particular function, saving time and reducing the risk of mistakes, and **2)** users do not have to manually define a particular workflow–a CHG will be generated and then *all* functions for which we have a complete set of inputs can be automatically executed. Future work will focus on improving the efficacy and efficiency of the solver, and exploring other potential applications.

References

Arp, R., Smith, B., & Spear, A. D. (2015). *Building ontologies with Basic Formal Ontology*. MIT Press.

Beverley, J., Logan, J., & Smith, B. (2025). Unreal Patterns. *arXiv preprint arXiv:2506.06284*.

Buffoni, L., Pop, A., & Mengist, A. (2017). Traceability and impact analysis in requirement verification. *Proceedings of the 8th International Workshop on Equation-Based Object-Oriented Modeling Languages and Tools*, 95–98.

Dertien, S., & Hastings, W. (2021). The State of Digital Thread. *PTC White Paper*.

Elaasar, M. (2019). OpenCAESAR Initiative. *JPL Model-Based Systems Engineering (MBSE) Symposium and Workshop*.

Gregory, J., Iyer, V., & Salado, A. (2025). Semantically Enabled Dashboards to Support Systems Engineers. *INCOSE International Symposium*, *35*(1), 1240–1256.

Gregory, J., & Morris, J. (2025). KG 2 CHG Github Repository. https://github.com/joegregoryphd/KG2CHG

Gregory, J., & Salado, A. (2024). An ontology-based digital test and evaluation master plan (dTEMP) compliant with DoD policy. *Systems Engineering*, *27*(6), 1012–1026.

Gregory, J., Salado, A., O'Neal, S., Larez, R., Reda, C., Martell, N., Martin, E., Colson, M., Masterson, J., & Armenta, D. (2024). The Digital Engineering Factory: Considerations, Current Status, and Lessons Learned. *INCOSE International Symposium*, *34*(1), 927–943.

Gregory, J. R., Iyer, V., Galves, A., Hoag, L., Marmar, M., Zute, A., Jones, B., & Salado, A. (2025). Digital Engineering for Cubesats With Semantic Web Technologies. *AIAA SCITECH 2025 Forum*, 1656.

Gruber, T. R. (1991). The role of common ontology in achieving sharable, reusable knowledge bases. *Principles of Knowledge Representation and Reasoning: Proceedings of the Second International Conference*, 601–602. http://www.cin.ufpe.br/$%5Csim$mtcfa/files/10.1.1.35.1743.pdf

Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, *5*(2), 199–220. https://doi.org/10.1006/knac.1993.1008

Hevner, A., & Chatterjee, S. (2010). Design science research in information systems. In *Design research in information systems: Theory and practice* (pp. 9–22). Springer.

Hevner, A. R. (2007). A three cycle view of design science research. *Scandinavian journal of information systems*, *19*(2), 4.

Karray, M., Otte, N., Rai, R., Ameri, F., Kulvatunyou, B., Smith, B., Kiritsis, D., Will, C., Arista, R., et al. (2021). The industrial ontologies foundry (IOF) perspectives.

Medhi, S., & Baruah, H. K. (2017). Relational database and graph database: A comparative analysis. *Journal of process management and new technologies*, *5*(2), 1–9.

Mengist, A., Buffoni, L., & Pop, A. (2021). An integrated framework for traceability and impact analysis in requirements verification of cyber–physical systems. *Electronics*, *10*(8), 983.

Morris, J. (2025). ConstraintHg: A Kernel for Systems Modeling and Simulation. *Under review with Journal of Open-Source Software*.

Morris, J., Indupally, A., Mocko, G., Wagner, J., & Ramnath, S. (2025). Declarative, Multiphysics Simulation Between Applications via Constraint Hypergraphs. *Under review with J. Comput. Inf. Sci. Eng.*

Morris, J., Mocko, G., & Wagner, J. (2025). Unified System Modeling and Simulation via Constraint Hypergraphs. *Journal of Computing and Information Science in Engineering*, *25*(6), 061005. https://doi.org/10.1115/1.4068375

OpenCAESAR. (2023). Ontological Modeling Language 2.0.0 [Available at http://www.opencaesar.io/oml/, Date Accessed: 2025-09-25].

Otter, M., Thuy, N., Bouskela, D., Buffoni, L., Elmqvist, H., Fritzson, P., Garro, A., Jardin, A., Olsson, H., Payelleville, M., et al. (2015). Formal requirements modeling for simulation-based verification. *Proceedings of the 11th International Modelica Conference, Versailles, France*.

Patel, A., & Jain, S. (2021). Present and future of semantic web technologies: a research statement. *International Journal of Computers and Applications*, *43*(5), 413–422.

Pérez, J., Arenas, M., & Gutierrez, C. (2009). Semantics and complexity of SPARQL. *ACM Transactions on Database Systems (TODS)*, *34*(3), 1–45.

SymPy Development Team. (2025). SymPy. https://www.sympy.org/en/index.html

Wagner, D., Kim-Castet, S. Y., Jimenez, A., Elaasar, M., Rouquette, N., & Jenkins, S. (2020). CAESAR Model-Based Approach to Harness Design. *2020 IEEE Aerospace Conference*, 1–13.

Willems, J. C. (2007). The Behavioral Approach to Open and Interconnected Systems. *IEEE Control Systems Magazine*, *27*(6), 46–99. https://doi.org/10.1109/MCS.2007.906923

World Wide Web Consortium. (1997). Resource Description Framework (RDF) Model and Syntax.

World Wide Web Consortium. (2013). SPARQL 1.1 Query Language.

Zimmerman, P., Gilbert, T., & Salvatore, F. (2019). Digital engineering transformation across the Department of Defense. *The Journal of Defense Modeling and Simulation*, *16*(4), 325–338.

Biography

Author Name
Provide a short biography of the author.
Provide a short biography of the author.

Second Author
Provide a short biography of the second author.

Third Author
Provide a short biography of the third author.